Design and Analysis of Algorithms
(E1ITB303)

Micro Project Report

On

Smart Route Finder

Submitted by

Y. Yateesh Kumar Reddy	2411021060654
M. Santosh	2411021060397
Y. Shashank	2411021060891
P. Vamsi Chowdary	2411021060894

Table of Contents

Abstract

1. **Introduction**
   1.1 Project Overview
   1.2 Objectives of the Project
   1.3 Problem Statement
   1.4 Scope of the Project
2. **Algorithm Design**
   2.1 Description of the Algorithm
   2.2 Flowchart
   2.3 Pseudocode
3. **Complexity Analysis**
   3.1 Time Complexity (Best, Average, Worst Case)
   3.2 Space Complexity
4. **Implementation**
   4.1 Programming Language Used
   4.2 Code Explanation
5. **Results and Discussion**
   5.1 Output
   5.2 Performance Analysis
6. **Conclusion**
7. **Future Scope**

References

Abstract

The Smart Route Finder project is designed to determine the shortest path between multiple locations using graph-based algorithms. In this project, each location is represented as a node (vertex) and the connections between them are represented as edges with weights indicating distance or cost. The project uses Dijkstra's Algorithm, which is a greedy algorithm, to compute the shortest path from a selected source node to all other nodes efficiently.

This algorithm ensures optimal path selection by always choosing the minimum distance node and updating its neighboring nodes. The system can be applied in real-world scenarios such as GPS navigation systems, delivery route optimization, and communication networks. The project provides a practical understanding of graph theory, algorithm efficiency, and optimization techniques.

1. Introduction

1.1 Project Overview

Smart Route Finder is a system that finds the shortest path between locations using efficient algorithms. It simulates real-world navigation systems by modeling locations and roads as a graph structure.

1.2 Objectives of the Project

- To implement Dijkstra's shortest path algorithm
- To understand graph representation using adjacency lists
- To analyze algorithm efficiency and complexity
- To apply algorithms to real-world problems
- To improve logical and analytical thinking

1.3 Problem Statement

In real-world scenarios, finding the shortest path between locations is a common problem. Given a network of locations connected by roads with distances, the goal is to find the minimum distance path from a source location to all other locations.

1.4 Need for the Project

- Helps reduce travel time and fuel consumption
- Improves efficiency in logistics and delivery
- Useful in network communication systems
- Provides optimized solutions in real-time applications

1.5 Scope of the Project

- Navigation systems (Google Maps-like applications)
- Transportation and logistics
- Network routing in computer science
- Emergency services route optimization

2. Algorithm Design

2.1 Introduction to Dijkstra's Algorithm

Dijkstra's Algorithm is a greedy algorithm used to find the shortest path between nodes in a weighted graph. It guarantees the shortest path when all edge weights are non-negative**.**

2.2 Working Principle

The algorithm works by:

- Assigning initial distances
- Selecting the minimum distance node
- Updating neighboring nodes
- Repeating until all nodes are processed

2.3 Key Concepts

- Graph Representation
  - Nodes → Locations
  - Edges → Roads
  - Weights → Distance

- **Greedy Approach**

Always selects the best (minimum) option at each step

- **Relaxation**

New Distance = Current Distance + Edge Weight
If new distance is smaller → update

- **Priority Queue**
  - Used to select minimum distance node efficiently
  - Improves performance

2.4 Step-by-Step Example

Graph:

```
A --4-- B
|       |
2       5
|       |
C --8—D
```

Execution:

Step	Node	Update
1	A	B=4, C=2
2	C	D=10
3	B	D=9
4	D	Final

2.5 Pseudocode

```
Set all distances = infinity
Set source = 0

While queue not empty:
  Select minimum node
  Update neighbors
  Repeat
```

3. Complexity Analysis

3.1 Time Complexity

- Best Case: O(V log V)
- Average Case: O(E log V)
- Worst Case: O(E log V)

3.2 Space Complexity

O(V) for storing distances and graph

4. Implementation

4.1 Programming Language Used

- Java

4.2 Data Structures Used

- HashMap → Graph storage
- Priority Queue → Minimum selection
- Arrays/Maps → Distance storage

4.3 Code

```
import java.util.*;
class SmartRouteFinder {
    static class Node implements Comparable<Node> {
        String vertex;
        int distance;
        Node(String vertex, int distance) {
            this.vertex = vertex;
            this.distance = distance;
        }

        public int compareTo(Node other) {
            return this.distance - other.distance;
        }
    }

    public static Map<String, Integer> dijkstra(Map<String, Map<String, Integer>> graph, String
start) {
        PriorityQueue<Node> pq = new PriorityQueue<>();
        Map<String, Integer> distances = new HashMap<>();

        for (String vertex : graph.keySet()) {
            distances.put(vertex, Integer.MAX_VALUE);
        }

        distances.put(start, 0);
        pq.add(new Node(start, 0));

        while (!pq.isEmpty()) {
            Node current = pq.poll();
            String currentVertex = current.vertex;

            for (Map.Entry<String, Integer> neighbor : graph.get(currentVertex).entrySet()) {
                String nextVertex = neighbor.getKey();
                int weight = neighbor.getValue();

                int newDist = distances.get(currentVertex) + weight;
```

```
            if (newDist < distances.get(nextVertex)) {
                distances.put(nextVertex, newDist);
                pq.add(new Node(nextVertex, newDist));
            }
        }
    }

    return distances;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    Map<String, Map<String, Integer>> graph = new HashMap<>();

    System.out.println("===== SMART ROUTE FINDER =====");

    System.out.print("Enter number of vertices: ");
    int n = sc.nextInt();
    sc.nextLine();

    System.out.println("Enter vertex names (Example: A B C D):");
    for (int i = 0; i < n; i++) {
        String vertex = sc.nextLine();
        graph.put(vertex, new HashMap<>());
    }

    System.out.print("Enter number of edges: ");
    int e = sc.nextInt();

    System.out.println("Enter edges in this format: source destination weight");
    System.out.println("Example: A B 4");

    for (int i = 0; i < e; i++) {
        String source = sc.next();
        String dest = sc.next();
        int weight = sc.nextInt();

        graph.get(source).put(dest, weight);
        graph.get(dest).put(source, weight);
    }

    System.out.print("Enter starting node: ");
    String start = sc.next();

    Map<String, Integer> result = dijkstra(graph, start);
```

```
        System.out.println("\nShortest distances:");
        for (String node : result.keySet()) {
            System.out.println(node + " : " + result.get(node));
        }

        sc.close();
    }
}
```

4.4 Explanation of Code

- Graph is stored using adjacency list
- Distances are initialized to infinity
- Priority queue selects minimum node
- Distances are updated using relaxation
- Final distances are printed

5.Results and Discussion

5.1 Sample Input

```
===== SMART ROUTE FINDER =====
Enter number of vertices: 4
Enter vertex names (Example: A B C D):
A
B
C
D
Enter number of edges: 4
Enter edges in this format: source destination weight
Example: A B 4
A B 4
A C 5
C D 7
D B 9
Enter starting node: A
```

5.2 Output

```
Shortest distances:
A : 0
B : 4
C : 5
D : 12
```

5.3 Discussion

- The algorithm correctly finds shortest paths
- It avoids unnecessary calculations
- Efficient for real-time applications
- Works well for graphs with positive weights

6.Conclusion

The Smart Route Finder project successfully demonstrates how Dijkstra's Algorithm can be used to find the shortest path between different locations in a graph. The project models real-world scenarios by representing locations as nodes and roads as weighted edges. By applying a greedy approach, the algorithm efficiently calculates the minimum distance from a starting node to all other nodes.
This project helps in understanding important concepts such as graph representation, shortest path algorithms, and optimization techniques. It also shows how theoretical algorithms can be applied to practical applications like navigation systems and network routing. Overall, the project improves problem-solving skills and provides a strong foundation in algorithm design.

7.Future Scope

The Smart Route Finder project can be further enhanced in the following ways:

- Add real-time traffic data to calculate dynamic shortest paths
- Implement advanced algorithms like A* for faster performance
- Develop a Graphical User Interface (GUI) for better user interaction
- Integrate with Google Maps API or other mapping services
- Extend the system to support multiple destinations
- Add path visualization instead of only distances
- Modify the system to handle directed graphs and real road networks

References

1. Thomas H. Cormen, *Introduction to Algorithms*, MIT Press
2. GeeksforGeeks – Dijkstra's Algorithm
   https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm/
3. Oracle Java Documentation
   https://docs.oracle.com/javase/